

# Sorting-Based Clustering

## Applied technique via Genetic Algorithm

Luiz Fernando

June 21, 2026

### Abstract

Comma-Separated Values (CSV) is a list of elements separated by lines with values separated by commas. These files represent one of the usual dataset types that is used for clusterizations. For his ancient and important problem, we proposed the usage of ordering and Top-K Selection, this was further researched and understood as Sorting-Based Clustering. However, our implementation is aimed to work specifically with Genetic Algorithms (GA), therefore our implementation required a form of dynamism and flexibility necessary for the GA to properly work. In this work, we initially utilized a number of 20 generations and 40 individuals, with an Elitism of 5 members and mutation rates of up to 25%, while copying all genes from Top-5 individuals to all others. After this proposed, we received underwhelming results that prompted us to develop a second version of the GA, capable of solving the datasets presented with satisfaction. We found that while the method worked well, we noticed that our method struggled with datasets with vast quantities of parameters, a struggle that has been discussed in the literature for this approach. However we were still happy with the results of our method, and believe that with a couple of weeks it could potentially become viable for real use cases.

## 1 Introduction

For this project, I first envisioned the usage of my background (that of graph theory, percolation and Top-K Selection). My initial idea came from the usage of Top-K Selection to sort the list of elements in a dataset and then use this order to define clusters and segmentation.

This idea wasn't far off from actual researched topics. From my initial research I discovered two methods developed in the literature that utilized this very same concept for clustering of data.

The first, and more restrictive study, is Serialization, where elements are seen as vertices and their euclidian distance is used to set an order. The two closest vertices to each other are positioned as "neighbors" in the order.

The second method, the one that aligned better with our proposed, was a generalization of Serialization, where instead of euclidian distances, vertices were ordered according to distinct criterias.

In both cases, clustering is dynamically determined during runtime, this was also hypothesized by our team during research, as the concepts from Top-K Selection could be used here to dynamically guess clustering segmentation.

The idea behind is to select vertices into clusters until the difference between two indexes is considered bigger than a set threshold. However our team had issues with this approach, as we found that the threshold did not serve a sufficiently smart form of neighbor segmentation.

While clustering can be very complex utilizing many filter to increase performance and precision [CSWY24] our method will remain simple, while also innovating.

As this project was developed with the intent to work inside a Genetic Algorithm (GA) pipeline. This is, to the best of our knowledge, a research field that is still underdeveloped.

## 2 Objectives

Therefore, we propose the development of the second type of method talked in the introduction, known as Sorting-Based Clustering (SBC). This method utilizes distinct criterias determined by each project to order a list of data.

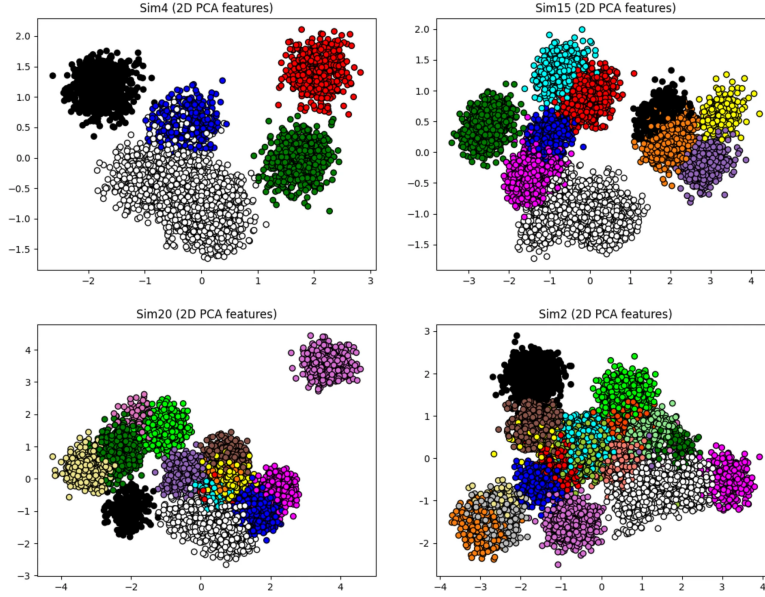


Figure 1: Expected results of a SBC with spike ordering [AP25].

## 2.1 Sorting-Based Clustering

SBC works in two simple stages, firstly the elements of the dataset are ordered according to a set criteria, then the distance among each vertex is calculated and any aberrant difference is set as a new cluster, Figure 1 showcases a working method from the literature.

For this work, we propose the usage of individuals as parameters for this ordering, each individual contains a single integer byte for each value of the dataset. These parameters are used to order the dataset and further to determine clusters, centroids, and other metrics.

*Fitness* is then defined as the distance of each vertex to its respective cluster centroid, the sum of these distances is given as fitness of the individual. Therefore, obtain an individual's fitness he must calculate an order for the dataset, the clusters, their centroids, and finally the sum distance of each vertex. During our testings this expensive combination presented itself as a major drawback from our method.

## 2.2 Threshold

To define clusters, a threshold is decided to multiply the average distance among vertices. After this limit is established, any vertex that has an euclidian distance further than the limit set by Threshold will be separated into a new cluster.

We decided to keep this threshold as a fixed value, our understanding was that Individuals already had a vast amount of data to genetically go through, and since the threshold is capable of completely change the landscape that our Genetic Algorithm (GA) is trying to solve, leaving it at a fixed value initially appeared as a promising hypothesis.

This value was initially set as 3 times the average distance among vertices, however certain datasets required different values for any viable information to manifest.

Empirical testing proved that this threshold proved itself as a further hindrance to proper functionality. As we noticed that a simple number was not sufficient to determine clusters in a satisfactory manner, were sometimes no clusters were defined, while changing the number slightly would cause hundreds of clusters to appear.

Therefore we'd like to propose the usage of further techniques to segment the list of elements, by the utilization of further filters and smarter and more complex separations.

### 3 Genetic Algorithm

The Genetic Algorithm (GA) proposed here is, to the best of our knowledge, an innovative form of Sorting-Based Clustering (SBC). For this we had to translate the concepts of SBC to become dynamic and flexible, being used as parameters for our Individuals.

#### 3.1 Chromosome

As mentioned, an Individual's chromosome is a simple list of integers, this list represent the weights of values from the dataset. However, for the second version of the algorithm we decided to also allow the *Threshold* to be a value in the chromosome. In either case, the chromosome would be a simple list of integers, of up to  $1 + k$  bytes (where  $k$  is the number of parameters and *Threshold* is being represented by 1).

#### 3.2 Population

For the generational process, we chose 20 generations as an arbitrary number, as well as 40 individuals, these elements would be randomly assigned values for their parameters as the initial population is generated.

Our hypothesis was that – after seeing initial tests without GA – this was a problem that did not require a big amount of tries from the algorithm, however we would be proven wrong, when the first GA presented in underwhelming results.

#### 3.3 Mutations

For this work, we created three mutation methods, and eventually decided maintain only one. Complete randomness was initially created as a means of testing the GA, but was quickly dropped with the development of splice and mutations. Splice was briefly considered, however, by understanding the problem, we realized that splice would not be able to properly mutate over good individuals. Therefore the only viable solution that we found was simple mutation.

For this proposed we utilized the Elitism value of 5, meaning that from each generation the Top-5 members would propagate, while the rest of the individuals would become their copies, being allowed to mutate up to 25% of the copied values. This method, however, also changed for the second version of the algorithm.

At the end of the first version we realized that we need a good balance between exploitation and exploration, and for that we developed a mix between complete randomness and continues evolution of the best candidates.

#### 3.4 Fitness

Finally, the fitness of individuals was initially described as the euclidean distance of each vertex to the centroids of their respective cluster.

$$Fit(list, clu, cen) = \sum_{i=1}^{|list|} euclidian\_dist(list[i], cen[clu[i]]) \quad (1)$$

Where *clu* is the list representing the correspondence of a vertex and their cluster, and *cen* a smaller list containing the actual coordinates of centroids.

This showed good initial results, especially for simple datasets with low amount of vertices. While we believe better alternatives are possible, we accepted the behavior attached to this fitness as a good solution for our GA.

### 4 Experimental testings

To test our proposed method, we downloaded a couple datasets from the internet, with the idea of testing our method's performance and solutions.

Table 1: Summary of the datasets used in the clustering experiments.

| Dataset              | Attributes | Instances |
|----------------------|------------|-----------|
| Iris                 | 3          | 150       |
| Breast Cancer        | 31         | 569       |
| Mushrooms            | 22         | 8124      |
| Adult(Census Income) | 14         | 48842     |

As seen in Table 1, we’ve made use of the classical ”Iris” dataset but also other public datasets that are considered hard challenges for this type of problem. ”Breast Cancer” is a non-classifiable dataset, their inclusion was a way of seeing patterns of data without trying to define it, but also as a test of robustness when we employ high amounts of parameters. ”Mushrooms” is a vast dataset containing mushroom properties, it can be classified between poisonous and non-poisonous, however it is considered extremely challenging for clustering methods. Finally ”Adult” is a dataset focused on discovering if an individual of the data makes more or less than 50k dollars.

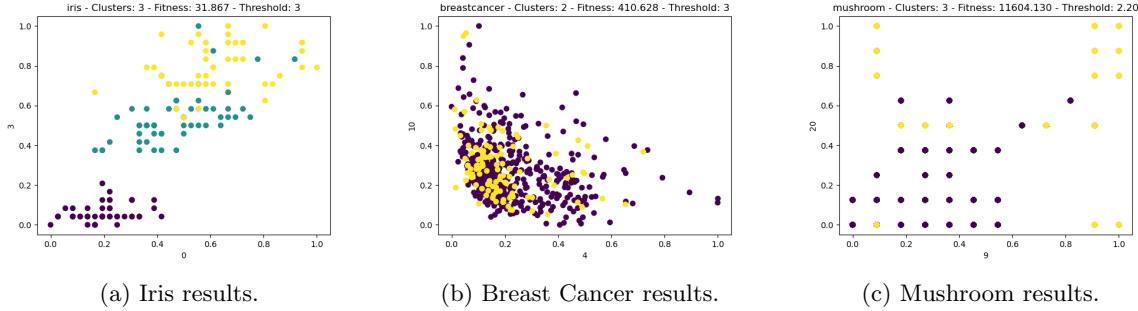


Figure 2: Results without the utilization of GA.

We initially tested these datasets without the implementation of GA, as seen in Figure 2. This test operated by simply sorting the dataset as is, then separating and clustering the elements with a custom threshold. This immediately returned with promising results, and we believed that the genetic algorithm would be the same.

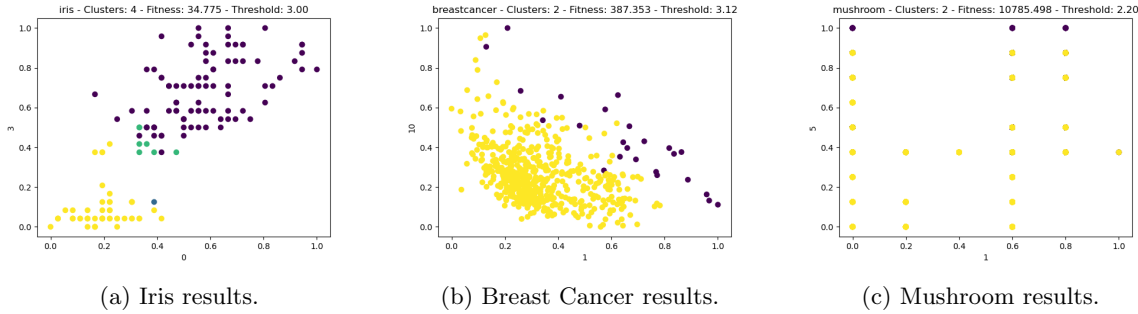


Figure 3: Results *with* the utilization of the first version GA.

We then further developed our GA methods and ran these datasets through them, with results found in Figure 3, attaining worse results than the benchmark established in Figure 2.

We believe that this unexpected worse result was due to the inefficient and still early stages of development of the GA, with fine tuning still required. With the main reasons for failure being pointed towards the usage of fixed thresholds, lack of diversity in generations and datasets with vast quantities of parameters, something we were aware was a problem for SBC [Ass12].

Through the results of the ”mushroom” dataset, we understood that the logic behind our fitness could be considered malformed. Due to the highly symmetrical nature of Mushroom, a centroid that resides close to the center of the values would result in a mostly small sum of distance. Therefore we

would require a distinct and smarter way of acquiring our fitness.

Moreover, from these tests we noticed that the best individuals of generations had little difference among each other, coinciding with our assumption that this problem would require more diversity.

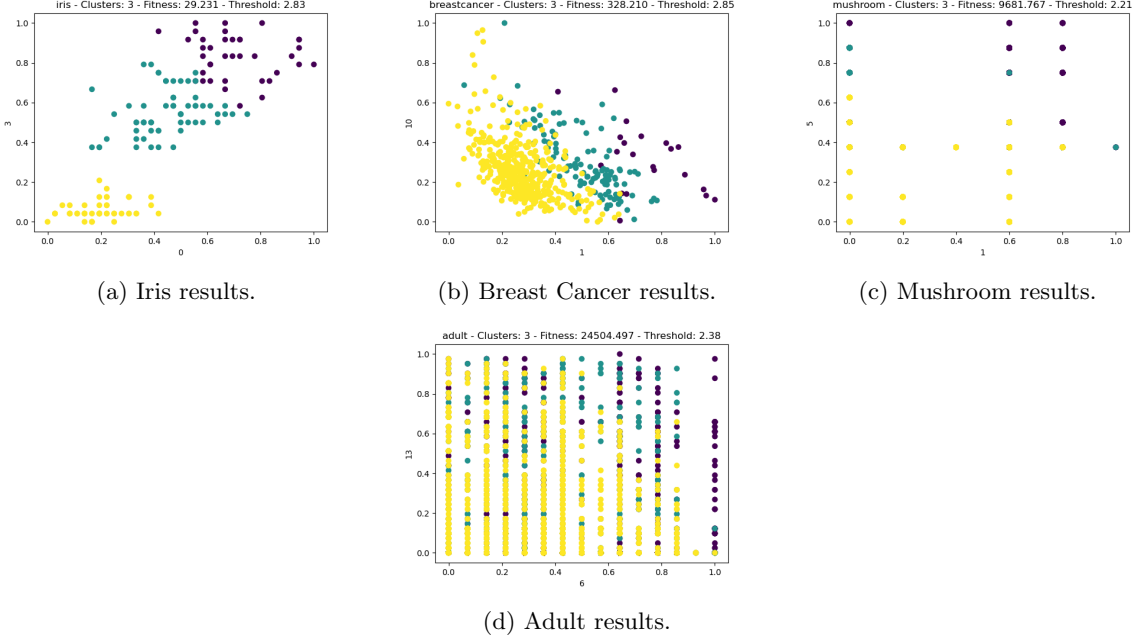


Figure 4: Results *with* the utilization of the second GA.

After fine-tuning, a second version of the GA was developed, with promising results seen in Figure 4. This second version employed more individuals per generation and twice as many generations as before. This choice more than doubled our processing time, however it resulted in far better results.

Not only that, but mutations were also altered to increase general diversity. In this version the gene from the *Top-Elitism* would only be copied to the following 5 individuals of the generation (matching amount to *Elitism*), while all other elements would have their parameters randomized. This is a combination of the best of both worlds, where we test different ideas for solutions while also working on the best ones.

Finally, the last change to the GA was allowing individuals to store and manipulate their thresholds. This means that each individual would contain  $1 + k$  bytes, where  $k$  is the number of values from the CSV. With this change, individuals dynamically altered their problem space to accommodate their weighted parameters, and naturally found the correct correspondences.

## 5 Conclusion

While very promising at first, Sorting-Based Clustering (SBC) methods of clusterization have a few limitations that are supported by the literature. They suffer heavily from datasets with great amounts of parameters [Ass12], and their two-staged nature means that their method is prone to be heavier and slower than alternatives.

The initial idea of fixing the threshold was sound in hypothesis, however in actual practice this manifested into a nightmarish problem, where threshold worked perfectly for "Iris" dataset, while they manifested completely broken for "mushroom", a dynamic threshold performed better, especially for the novel implementation of this method for Genetic Algorithm (GA).

Besides the threshold, other points of interest manifested problems in this work, such as the limited number of generations and individuals, as well as the small diversity allowed.

After fine-tuning and fixing, our proposed method performed greatly, albeit very slow, as seen in the last Figure 4. We achieved good solutions for simple datasets, and while we tested our method against non-ideal datasets, we were still able to generate meaningful clusters of information.

## References

- [AP25] Eugen-Richard Ardelean and Raluca Laura Portase. A study of deep clustering in spike sorting. *Neuroinformatics*, 23:51, 2025.
- [Ass12] Ira Assent. Clustering high dimensional data. *WIREs Data Mining and Knowledge Discovery*, 2(4):340–350, 2012.
- [CSWY24] Jiaqi Chen, Yaru Sun, Yang Wang, and Jie Yang. Deep clustering with distribution correction. *Pattern Recognition*, 149:110326, 2024.